

UFRPE-UAST

Esta é uma marca d'água para a versão de teste, compre para obter a versão completa.

Benefícios para usuários registrados:

1. Converte os documentos inteiros.

2. Nenhuma limitação para os documentos finais.

Celso Brennand

Remover Agora

Processos Distribuídos

AULA Nº 05

Os slides são baseados no livro Sistemas Distribuídos, de TANENBAUM, A. S. e STEEN, M. V.

Aula anterior, vimos:

- **Arquitetura descentralizada e arquitetura híbrida**

Conteúdo desta aula:

- **Revisitando processos e threads**
- **Threads distribuídas**
- **Processos clientes**
- **Processos servidores**

Processos

O que é Processo?

- no contexto de sistemas operacionais.

É um programa em execução!

- É a abstração de execução do SO (ou kernel).
 - Unidade de execução, de escalonamento, de endereçamento, de estado de contexto.
 - Programa se torna um processo (aka. job, task).
- Uma das principais tarefas do SO é gerenciar processos.
- O SO também é um processo.

Representando Processos

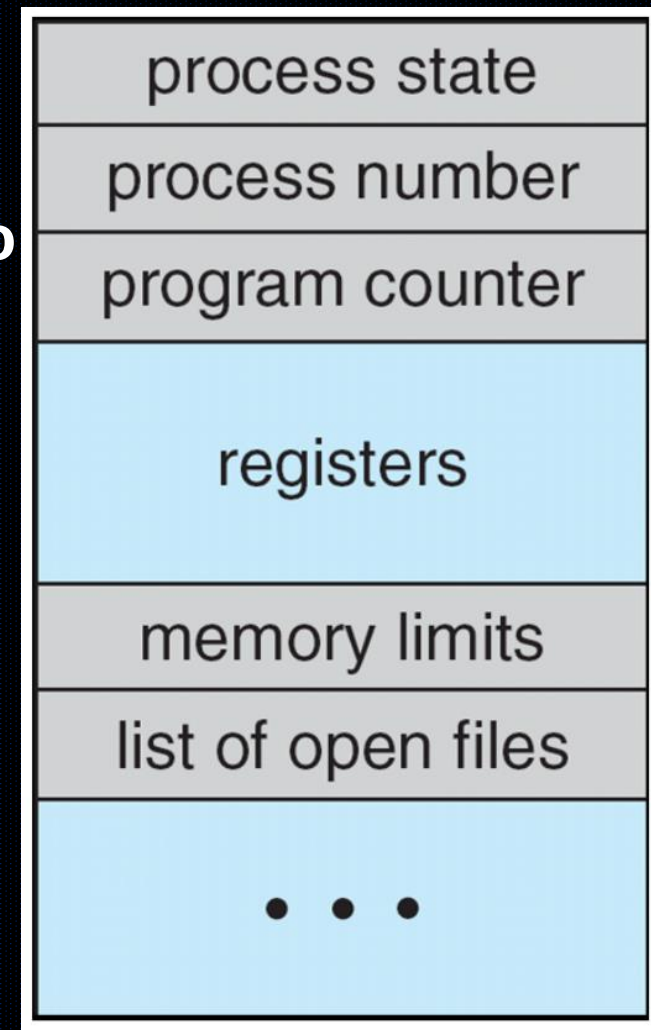
O que é necessário para o SO representar um processo?

Todo o estado de um programa em execução!

- Espaço de endereçamento (memória do processo)
- Código a ser executado (programa)
- Dados das variáveis do programa (estática e dinâmicas)
- Pilha de execução (chamada de funções)
- Contador de programa (PC)
- Registradores (como o PC)
- Recursos do SO sendo utilizados: descritores de arquivos, dispositivos de I/O, etc
- muitas outras coisas...

Process Control Block (PCB)

- Estrutura de dados mantida pelo SO para cada processo.
 - todo estado do processo.
 - inclusive valor dos registradores (da CPU) quando não executando.
- Armazenada em kernel space.
 - memória usada pelo SO.
- Grande quantidade de dados.
 - atualizada a “cada instante”.



Processos são pesados para o SO!

Process Control Block (PCB)

- Contexto de um Processo
- O **estado** do processo.
- Informações para escalonamento.
- Dados para contabilização de uso.
- Um **segmento de código**.
- Um **segmento de dados**.
- Os valores dos registradores.
- O **contador de programa**.
- Uma pilha de execução.
- Arquivos, portas e outros recursos alocados.

Troca de Contexto

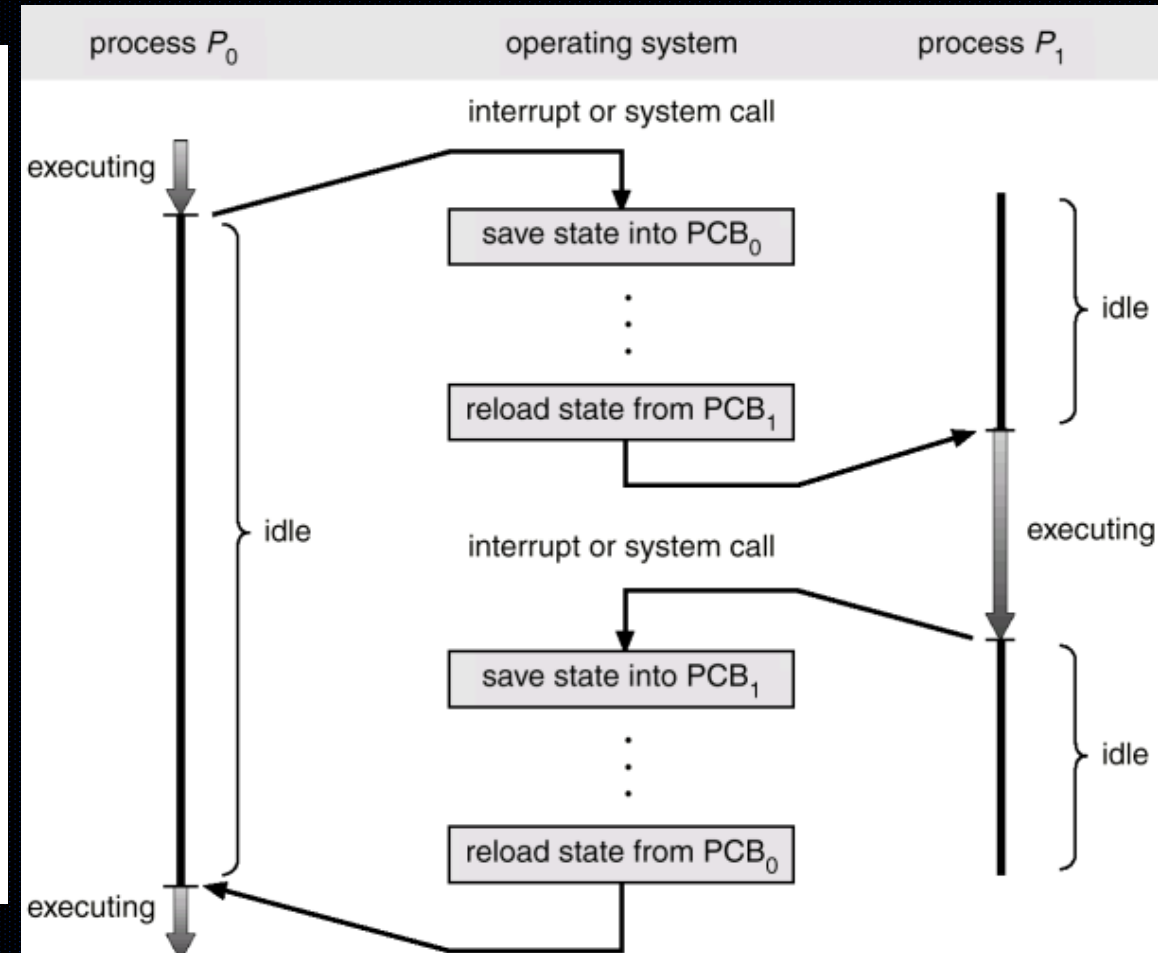
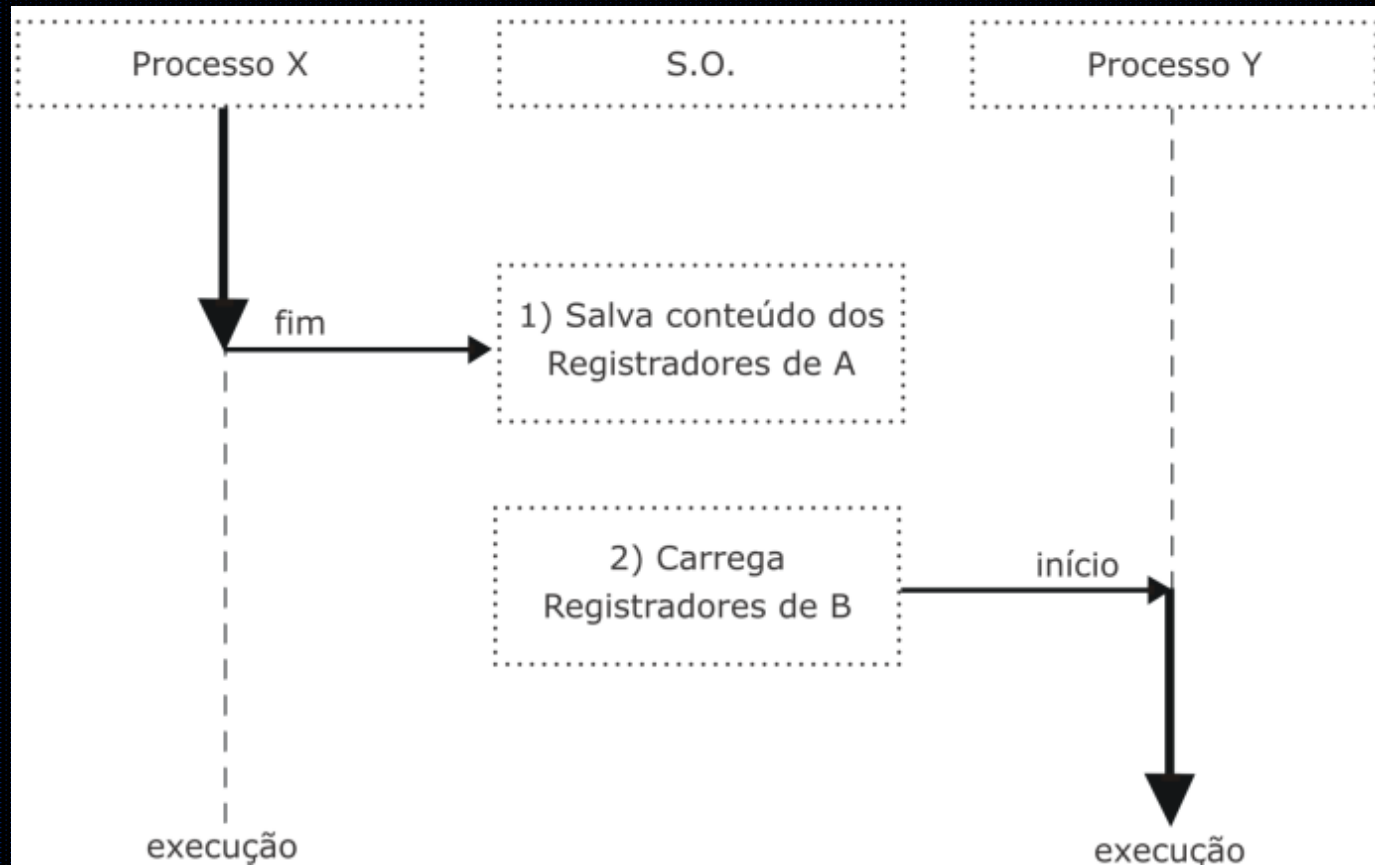
- O que é isto? Para que serve?
 - **Troca do processo que está em execução!**
- CPU (ou core) pode executar apenas um processo de cada vez.
- SO coloca e remove processos em execução.
 - ao remover, guarda “todo” o estado da CPU.
 - ao colocar, restaura “todo” o estado da CPU.
- Durante execução, PCB também é atualizado.
- Troca de contexto: 100 a 1000 vezes por segundo!

Realmente incrível!

Troca de Contexto

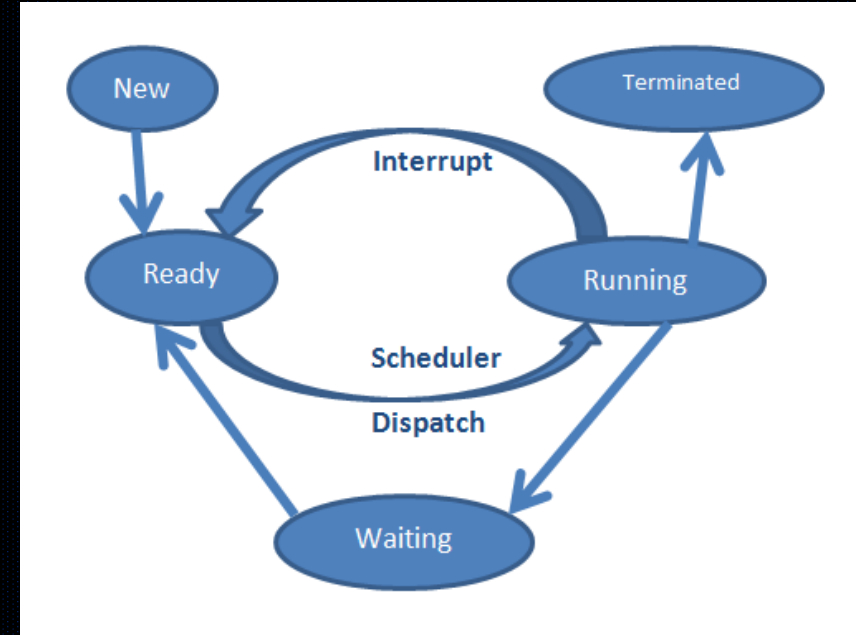
- Quando a CPU altera para um outro processo, o sistema deve armazenar o estado do processo antigo e carregar o estado armazenado do novo processo.
- O tempo gasto para a mudança de contexto não é útil aos processos (overhead).
- Este tempo gasto é dependente das características do hardware.

Troca de Contexto



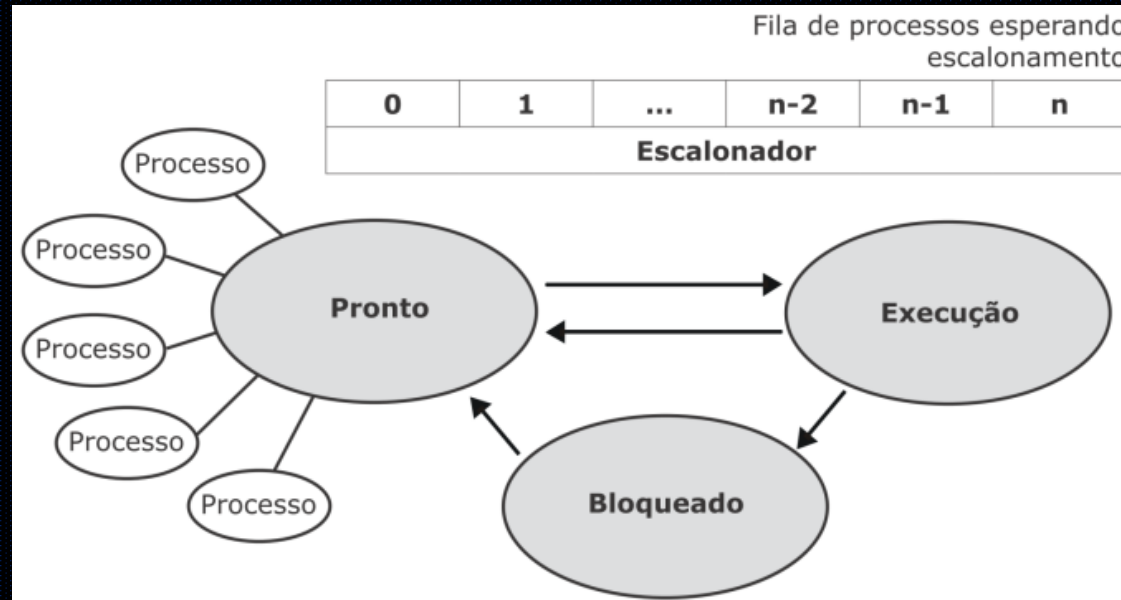
Estado do Processo

- Indica o que o processo está fazendo
 - Quais são os estados de um processo?
- **Running:** executando instruções na CPU
- **Ready:** aguardando para ser escalonado na CPU
- **Waiting:** aguardando algum evento para continuar execução (ex. usuário teclar alguma coisa)
- Em que estado um processo qualquer passa a maior parte do tempo? (Io-Waiting / batch-Ready)
- Quantos processos podem estar no estado Running? (igual #nucleos)
- Quantos processos um SO gerencia normalmente? (centenas)



Escalonamento

- Estado de Ready (pronto) possui uma fila de processos aguardando a liberação da CPU.



- O componente que tenta manter o processador ocupado o tempo todo, escolhendo que será o próximo processo a ser executado é o escalonador.

Escalonamento- Categorias de algoritmos

- **Não-preemptivos**

é quando um processo em execução não pode ser interrompido pelo S.O.

- First-in-First-out (FIFO): chegar primeiro é o primeiro a ser selecionado.
- Shortest-Job-First(SJF): Associa cada processo ao seu tempo de execução.
- Cooperativo: voluntariamente livra o processador.

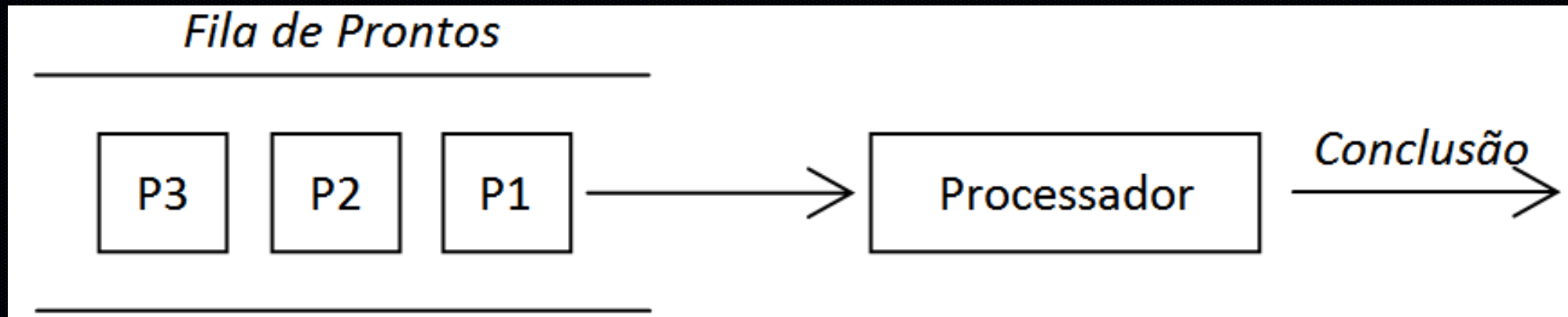
- **Preemptivos**

SO pode interromper um processo em execução e passa-lo para o estado de pronto

- Alternância Circular, por prioridades e por múltiplas filas
- Round Robin

Escalonamento- First In First Out (FIFO)

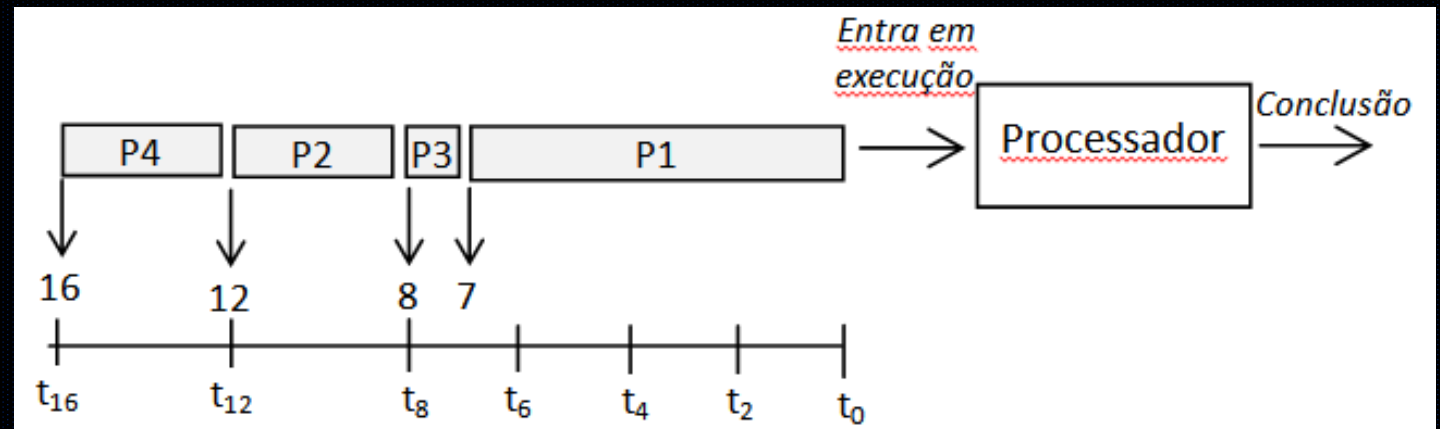
- Constitui-se no esquema mais simples de escalonamento em que os processos são executados do início até o fim, na ordem de chegada.



Escalonamento-Shortest Job First

- Processos menores são executados primeiro.
- Processos curtos são favorecidos.
- Processo maiores são prejudicados.
- Pouco utilizado na prática.

Processo	Tempo de Uso da CPU em ms (milissegundos)	Chegada
P1	7	0
P2	4	2
P3	1	4
P4	4	5

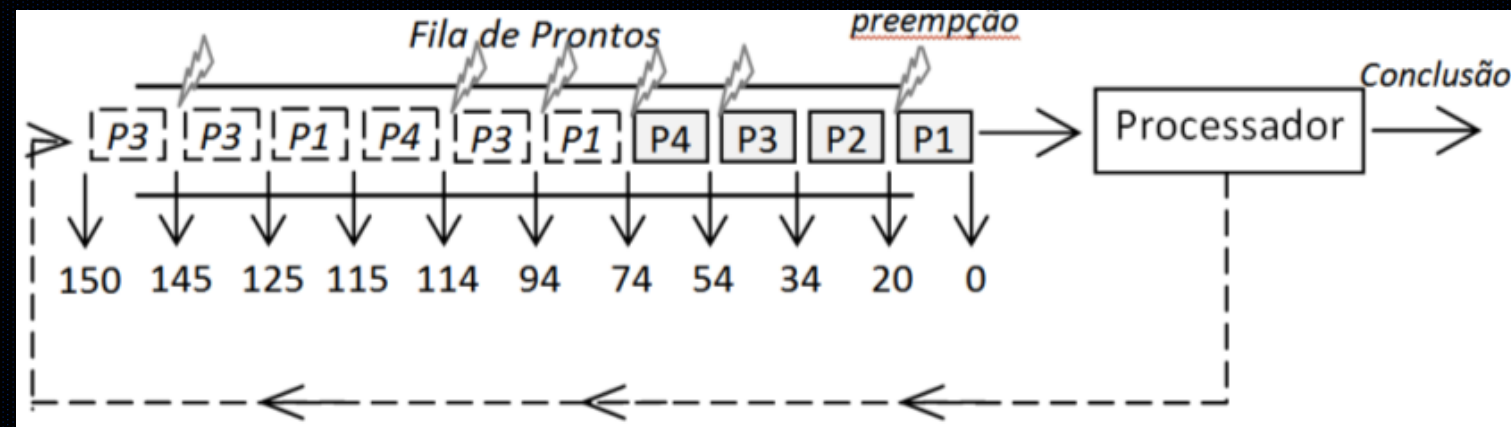


Escalonamento- Round Robin

- Processos são executados na ordem FIFO, mas com um intervalo chamado quantum.
- Ao final de seu quantum, se o processo ainda estiver em execução, é interrompido (preempção) e voltará para o estado de pronto (final da fila) e o próximo processo da fila será alocado para ocupar a CPU.
- Se o processo terminar antes de finalizar o seu quantum, a CPU será liberada.

Exemplo: *quantum* de 20 ms

Processo	Tempo de Uso da CPU em ms (milissegundos)
P1	50
P2	14
P3	65
P4	21



Chamada do Sistema



- **System call (syscall):** função do sistema operacional que um programa pode chamar
- **Mecanismo para oferecer serviços ao processo**
 - ex. ler do teclado, imprimir no terminal, transmitir dados pela rede, etc
- **Oferece abstração do serviço; permite compartilhar recurso entre processos**
 - ex. `print("Bom dia")`, `send(S, "Boa tarde")`
 - Linux: mais de 300 syscalls
- **Não induzem troca de contexto (troca de privilégio de execução, executam em kernel space)**
 - relativamente pesadas, e podem bloquear processo

Processos se Comunicando

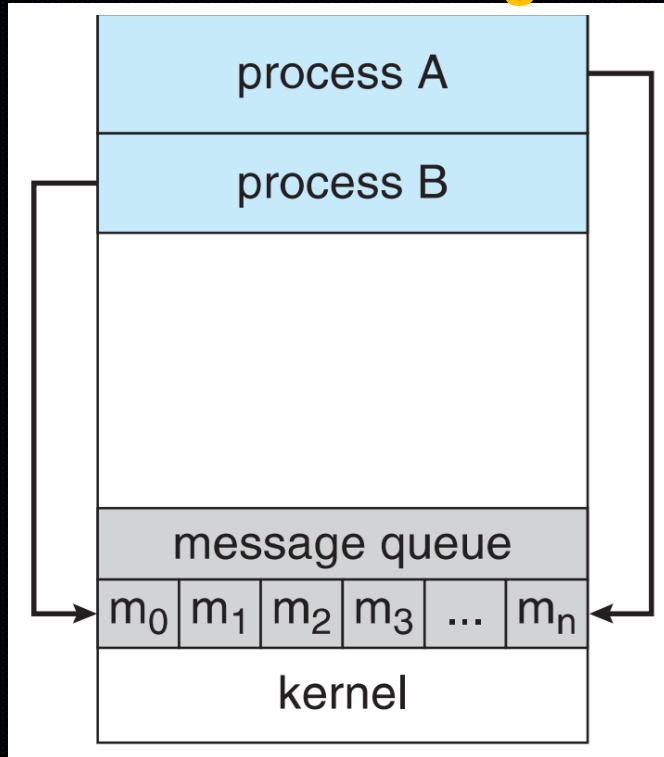
- **Processos muitas vezes precisam interagir**
 - para trocar informação ou coordenar
 - os processos são isolados pelo SO
- **SO precisa oferecer mecanismos de comunicação**

Interprocess Communication (IPC)

- **Dois modelos de comunicação**
 - memória compartilhada
 - troca de mensagens
- **Para cada modelo, diferentes técnicas e implementações**

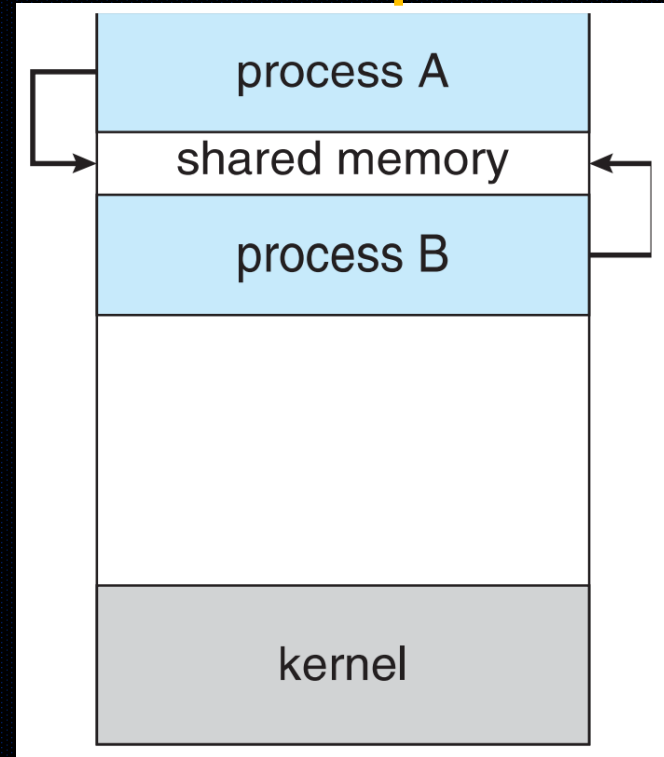
Modelos de IPC

Troca de mensagens



- Canal de comunicação explícito através de chamadas send/receive
- Fila de mensagens em kernel space

Memória compartilhada



- Região de memória mapeada em dois ou mais processos
- Memória fora do kernel space

Processos se Comunicando

- **Processos muitas vezes precisam interagir**
 - para trocar informação ou coordenar
 - os processos são isolados pelo SO
- **SO precisa oferecer mecanismos de comunicação**

Interprocess Communication (IPC)

- **Dois modelos de comunicação**
 - memória compartilhada
 - troca de mensagens
- **Para cada modelo, diferentes técnicas e implementações**

Memória Compartilhada

- Uma única chamada do sistema para definir e mapear a região de memória a ser compartilhada
- SO não faz mediação de leitura/escrita
- Processos precisam coordenar
 - o que acontece quando os dois querem escrever? (sincronização)

Fonte de grandes problemas!

- Coordenação através de sincronização
- Veremos nas próximas aulas

Troca de Mensagens

- **Processos trocam mensagens através de chamadas do sistema (system call): send/receive**
 - “enlace” de comunicação
 - não há memória compartilhada
 - chamada ao sistema para cada mensagem
- **Fila de mensagens em kernel space**
- **Características do enlace/troca de mensagens**
 - direto x indireto
 - síncrono x assíncrono
 - bufferização

Direto x Indireto

Direto

- Processo origem/destino
- precisa ser identificado
 - `send(P, msg)` : envia mensagem ao processo P
 - `receive(Q, msg)` : recebe mensagem do processo Q
- Enlace criado “automaticamente”
- Enlaces são unidirecionais
- Ex. sinais

Indireto

- Mensagens enviadas, recebidas através de uma “caixa postal” (CP)
 - `send(A, msg)` : envia mensagem para CP A
 - `receive(A, msg)` : recebe mensagem da CP A
- Processos precisam criar e se conectar a CP
- CP são bidirecionais
- Processos pode ter múltiplas CPs
- Ex. pipes, sockets

Síncrono x assíncrono

Síncrono

- Comunicação é **bloqueante** (blocking)
 - `send(A, msg)` : transmissor aguarda até o receptor receber
 - `receive(A, msg)` : receptor aguarda até mensagem chegar
- Aguardar = bloqueado: processo em estado Waiting
- Forte sincronia entre enviar e receber

Assíncrono

- Comunicação é **não bloqueante** (non-blocking)
 - `send(A, msg)` : transmissor envia e continua execução
 - `receive(A, msg)` : recebe mensagem caso exista (erro caso contrário), e continua execução
- Processos não aguardam = não ficam bloqueados

Exemplo: Produtor-Consumidor

- **Dois processos**
 - produtor: gera informação
 - consumidor: processa informação
- **Produtor envia informação ao consumidor**

Produtor

```
...  
message produced;  
while (true) {  
    produced = generate_item();  
    send(A, produced);  
}
```

Consumidor

```
...  
message consumed;  
while (true) {  
    receive(A, consumed);  
    process(consumed);  
}
```

- **O que acontece com cada caso send/receive bloqueante e não bloqueante?**

Bufferização

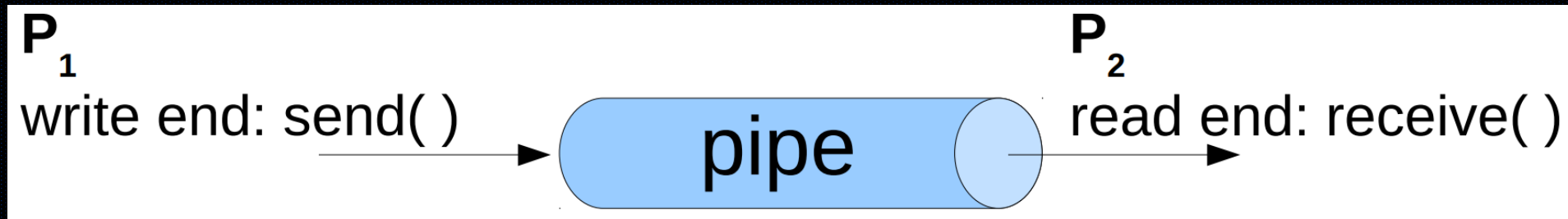
- Mensagens transmitidas podem ser armazenadas pelo “enlace” ou “canal”
 - tanto do lado do transmissor, quanto do lado do receptor (quando processos em SO diferentes)
- Armazenamento feito pelo SO, em kernel space (memória do SO)
- Tamanho do buffer é limitado, mas pode ser controlado pelo processo
- Permite comunicação assíncrona
 - “transmite” para o SO; processo continua; SO entrega mensagem
 - O processo pode enviar mensagens mesmo que o outro processo esteja inativo

Sinais (signals)

- Forma primitiva de IPC, usada principalmente pelo SO para se comunicar com processos
- Processo envia mensagem (sinal) diretamente ao outro, de forma assíncrona
- Processo que recebe tem sua execução interrompida para processar a mensagem (sinal)
 - signal handler : função que processa a chegada da mensagem (sinal)
- Mensagem possui apenas um número inteiro, com diferentes significados
- Ex. programa kill no unix envia sinais a processos

Pipes

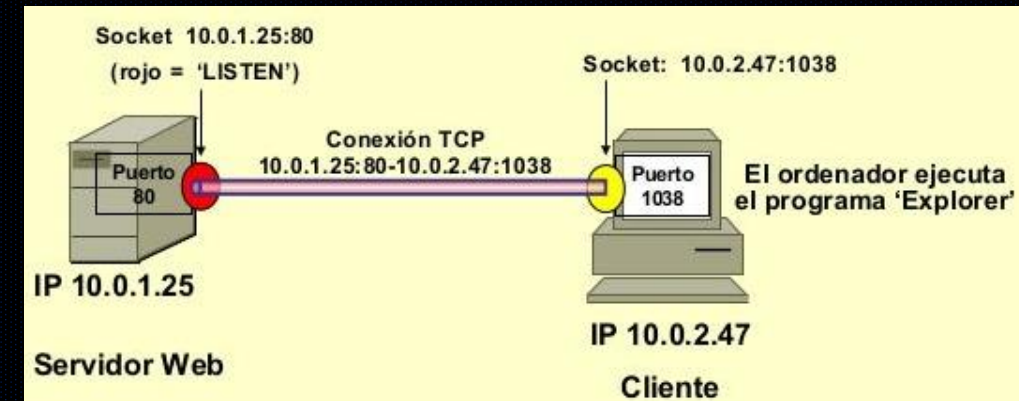
- **Forma Canal de comunicação (unidirecional) entre processos**
 - anonymous pipes, conduíte
- **Pipe tem dois lados**
 - write end: processo apenas escreve (send)
 - read end: processo apenas le (receive)



- **send()**: assíncrono quando há espaço no pipe, síncrono caso contrário (aguarda até ter espaço)
- **Ex. “|” no unix** cria um pipe entre o stdout do primeiro processo e o stdin do segundo
- **named pipe** oferece maior funcionalidade (múltiplos processos no mesmo pipe)

Sockets

- Mecanismo mais comum e poderoso para comunicação entre processos
 - base de todos os outros mecanismos, como RPC
- Permite comunicação entre processos de SO diferentes (computadores interligados por rede)
 - processo identificado pelo endereço IP e porta (número inteiro qualquer, acima de 1024)
- Dois tipos de enlace (sempre bi-direcional)
 - orientado a conexão (TCP): confiável, etc
 - sem conexão (UDP): não-confiável, etc
- API em todas as linguagens de programação



Revisitando processos e threads

Processo:

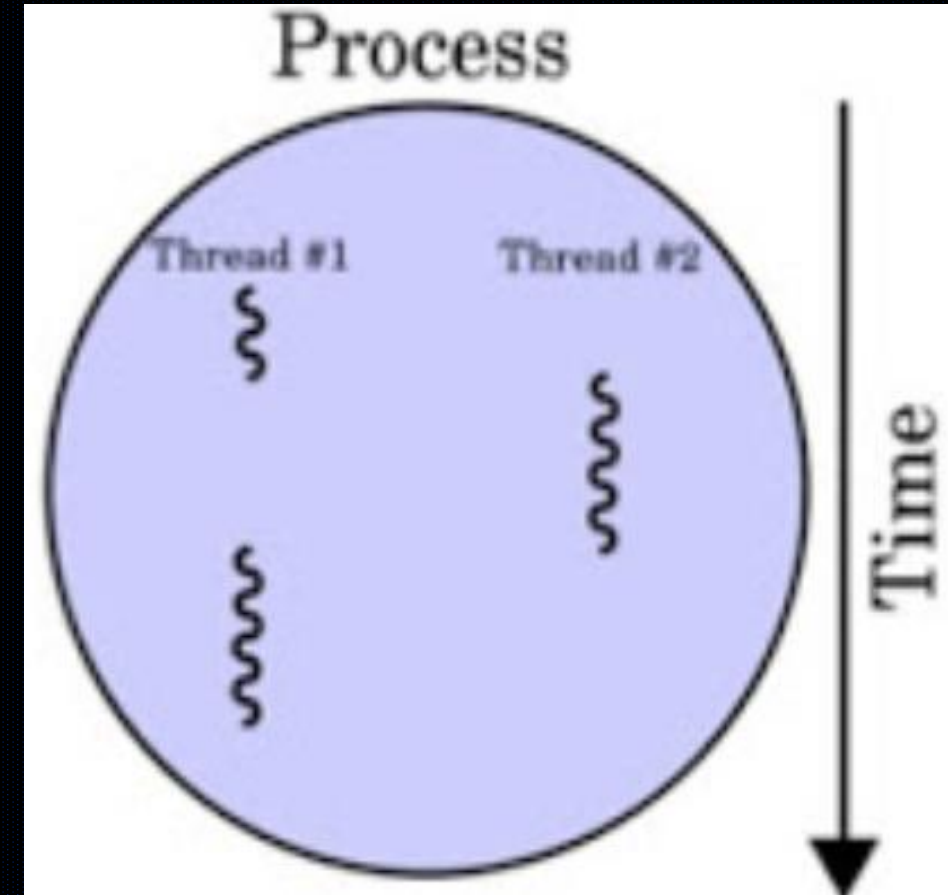
- Normalmente são independentes
- Espaços de endereçamento separados
- Interage com outros processos por meio de IPCs

Thread

- Fluxos de execução dos processos (subconjunto de processos)
- Compartilham o mesmo espaço de endereçamento e alguns dados da tabela de processo

Revisitando processos e threads

- Um processo servidor de arquivos **com um único fluxo** faz uma requisição do disco e espera pelo resultado.
- O mesmo servidor de arquivos com múltiplos fluxos pode atender a solicitações de outros usuários.
Aumento do throughput e do desempenho.



Fonte: Google Images

Processos e threads não-distribuídos

- Chaveamento de processo é um processo caro
- Tem que salvar e carregar as informações de contexto

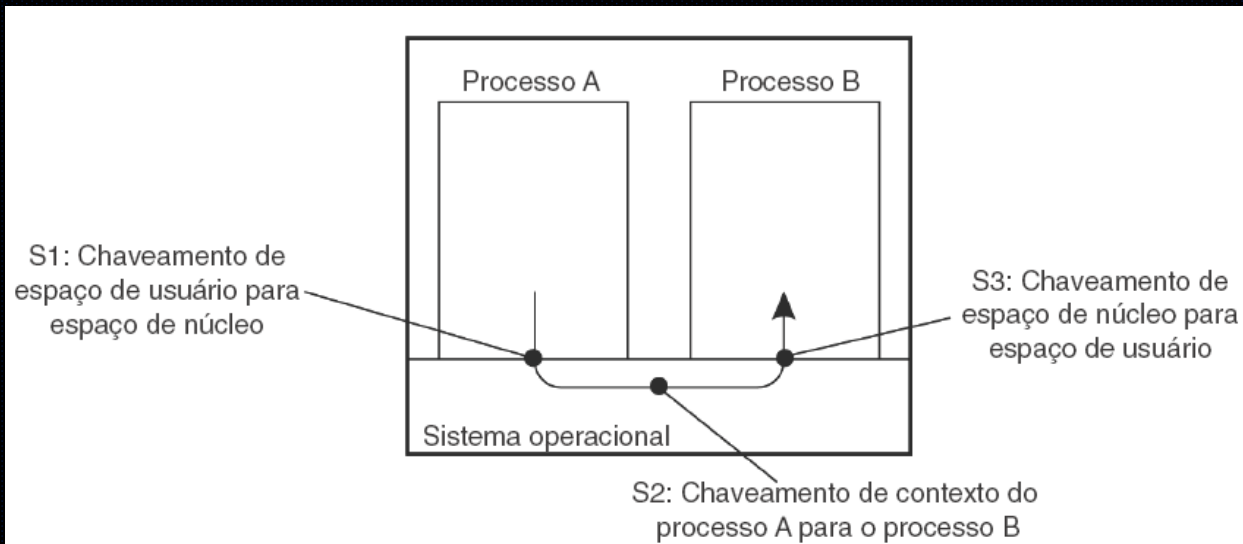
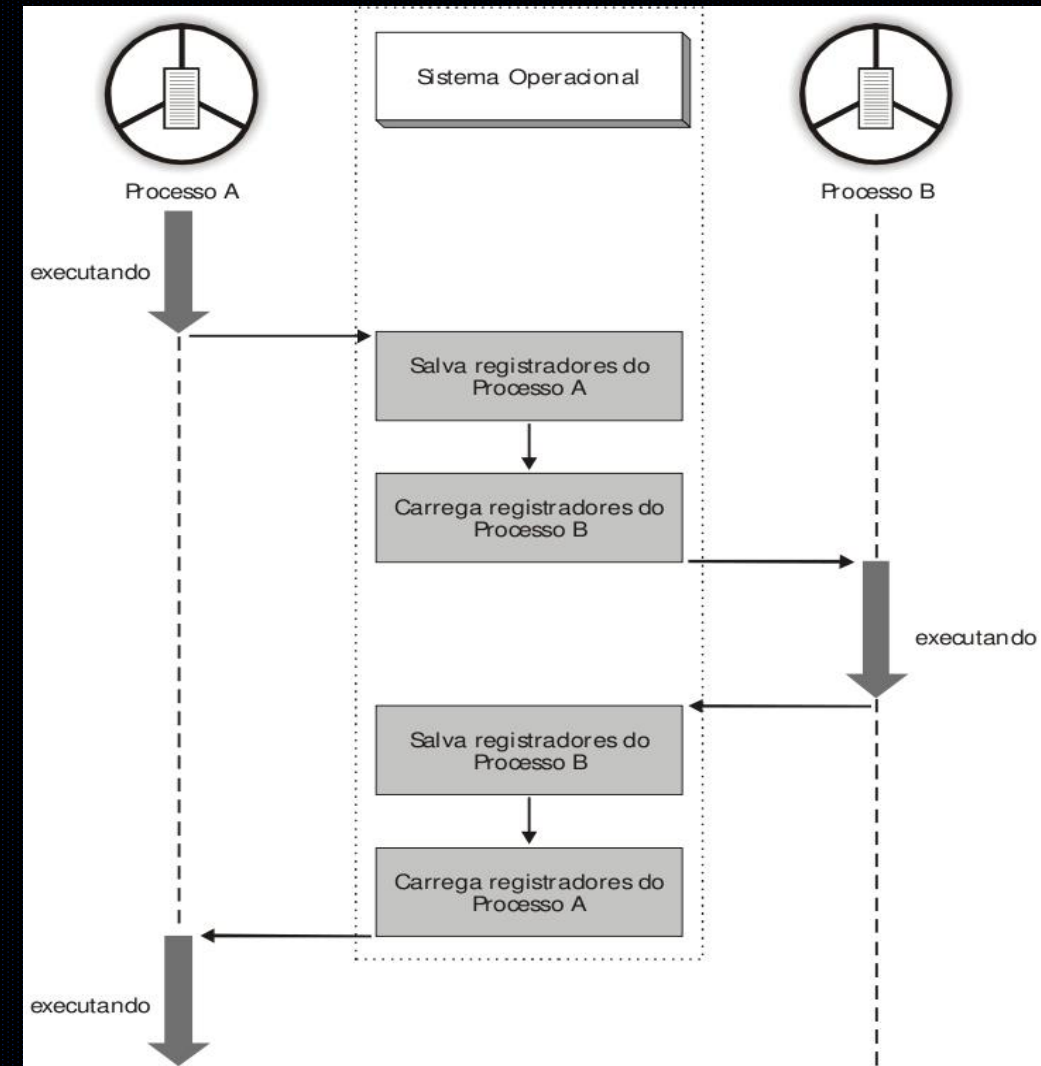


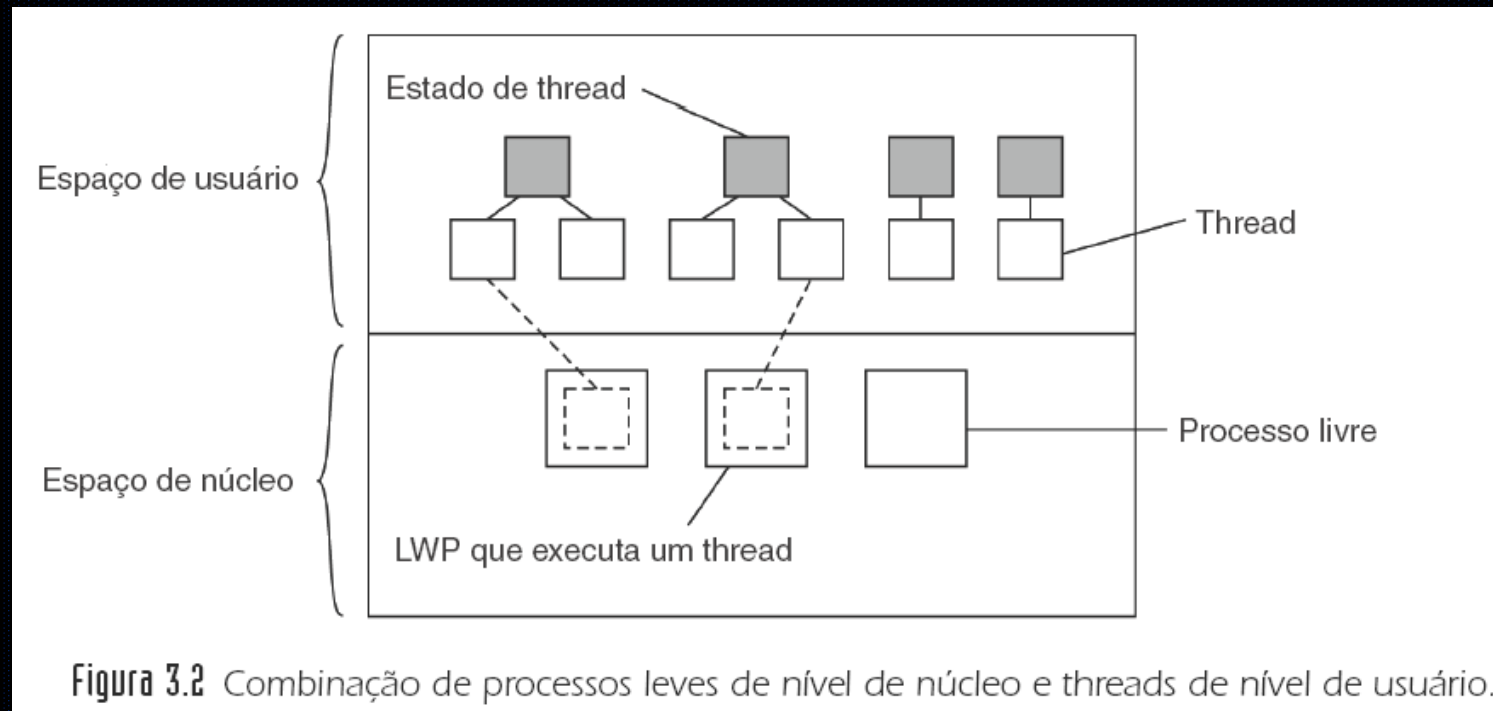
Figura 3.1 Chaveamento de contexto como resultado de IPC.



Processos e threads não-distribuídos

Threads Híbridas

- **Lightweith Process (LWP):** cada conjunto de threads é mapeada em um número n de threads do kernel.



Threads em sistemas distribuídos

Threads do Kernel permitem chamadas bloqueantes sem bloquear todo o processo.

- Vantagem das threads em SDs:
 - Permitem criar múltiplas conexões, cada uma sendo implementada por uma thread
 - Assim, **oculta-se a latência da comunicação na rede** (transparência)
 - Pode-se iniciar com a comunicação e logo depois realizar uma outra tarefa

Clientes Multithreads – Browsers Web

- Considerando uma conexão persistente com paralelismo
- Requisições são feitas sem que todos os objetos tenham chegado na máquina do cliente
- Cliente pode manipular diversos fluxos em paralelo usando as **Threads**
- Vantagem: usuário não necessita esperar até que todos os componentes da página cheguem (**transparência**)

Servidores Multithreads

- Como threads podem melhorar o desempenho dos servidores?
- **Funcionamento genérico dos servidores multithreads:**
 - Cada requisição que chega passa por uma thread despachante
 - Servidor escolhe um thread operário
 - O thread despachante pode ser selecionado para fazer o trabalho, caso todos estejam ocupados

Servidores Multithreads

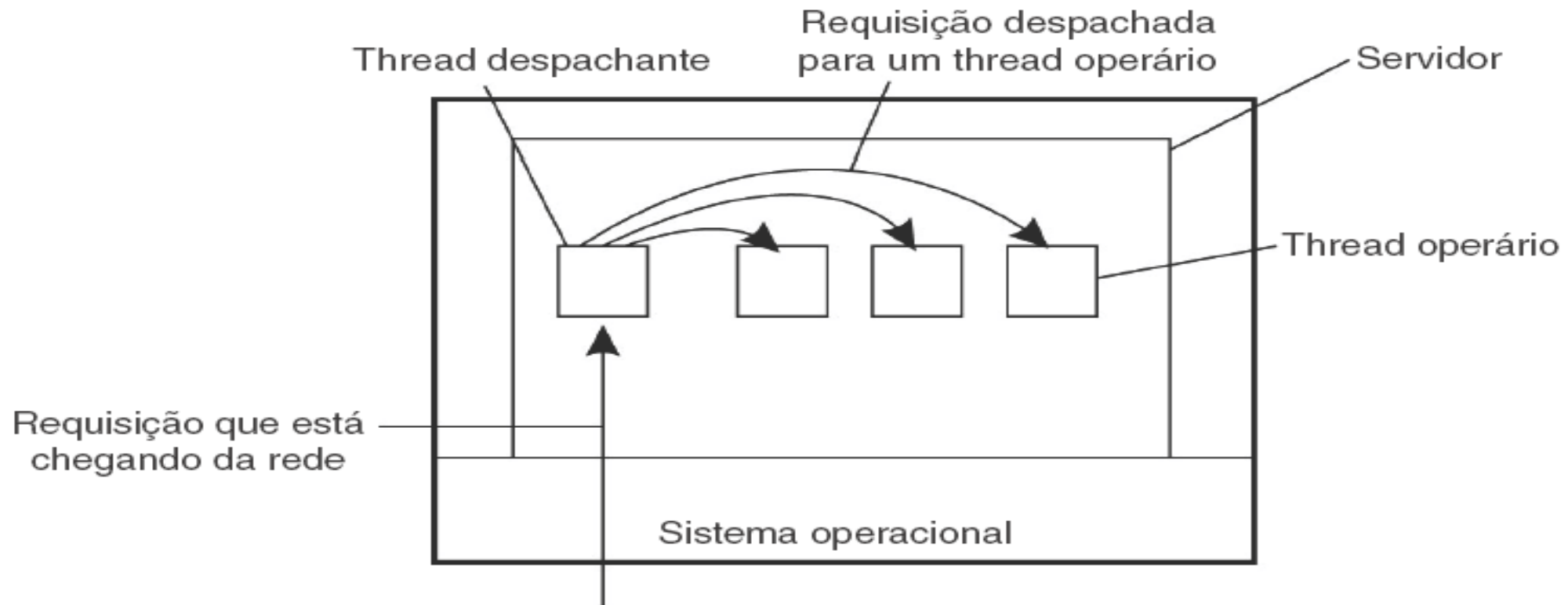


Figura 3.3 Servidor multithread organizado segundo modelo despachante/operário.

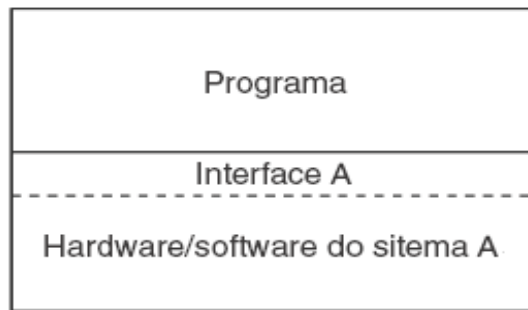
Virtualização

- Threads e processos – Dá a **ilusão** de fazer diversas tarefas ao mesmo tempo.
- Em computadores com uma CPU, a execução simultânea é uma **ilusão**:
 - Única CPU - somente uma thread ou processo será executada por vez
- Virtualização de recursos: “**fingir**” que um determinado recurso está **replicado** no sistema.

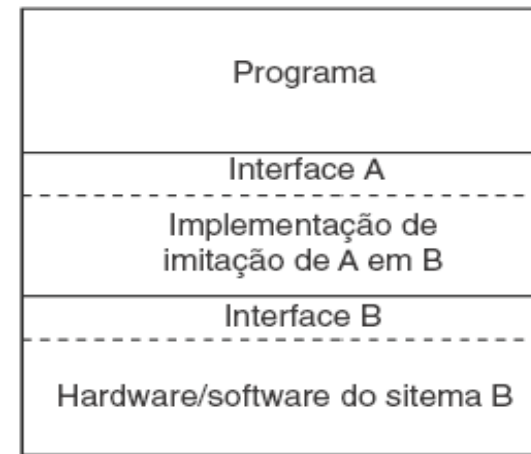
Virtualização

Os sistemas de virtualização:

- Estende ou substitui uma interface existente para imitar o comportamento de um outro sistema.



(a)



(b)

Figura 3.4 (a) Organização geral entre programa, interface e sistema. (b) Organização geral da virtualização do sistema A sobre o sistema B.

Virtualização na década de 1970

- A virtualização foi aplicada com sucesso nos mainframes IBM.
- Estes acompanhavam uma máquina virtual para executar softwares
 - Executar aplicações e SOs no IBM 370.
- Não é um conceito novo
- Até hoje essas VMs são utilizadas

Virtualização - Portabilidade

- Softwares em nível mais alto são mais estáveis (não mudam muito) do que o hardware e sistemas de software de baixo nível.
- Virtualização (middleware) pode ajudar transportando as interfaces de softwares para novas plataformas.
- Novas plataformas são capazes de executar softwares existentes anteriormente.

Arquiteturas de máquinas virtuais

- Existem quatro tipos e níveis diferentes de interfaces
- Níveis em que a virtualização pode ser implementada

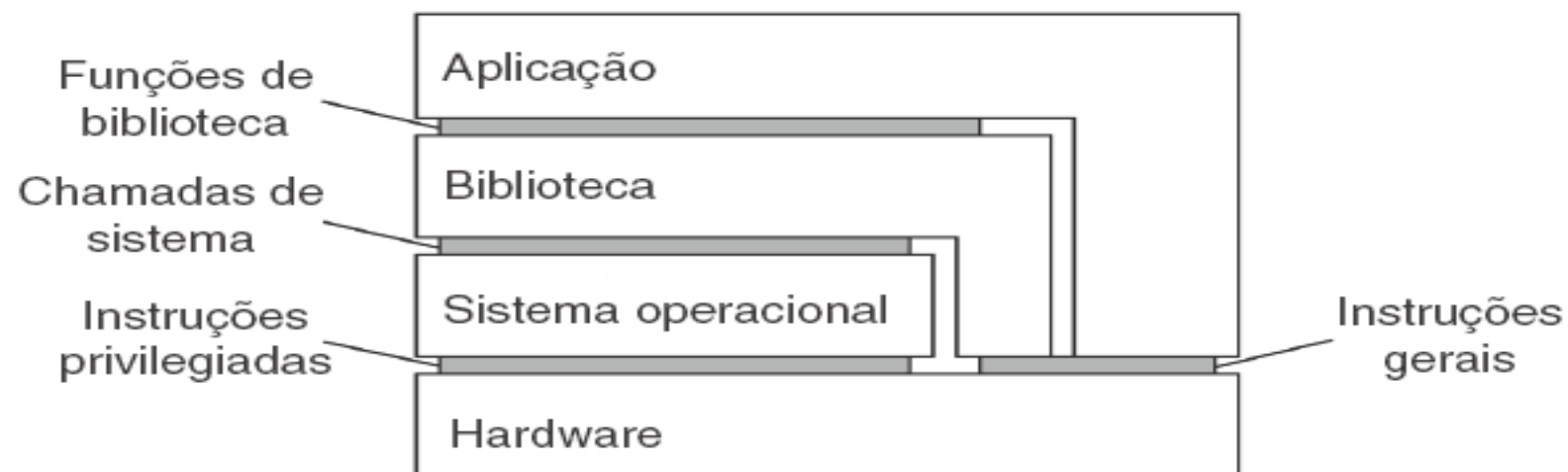


Figura 3.5 Várias interfaces oferecidas por sistemas de computadores.

Virtualização

Em resumo, é imitar o comportamento das interfaces (instruções de máquina, chamadas de sistema – *system calls*).

Dois tipos:

- Máquina virtual de processo
- Monitor de máquina virtual

Máquina virtual de processo

- Aplicações desenvolvidas para um SO são executadas em outro SO.
- Virtualização feita somente para **um único processo**.

Exemplo: *wine diretorio/application.exe*

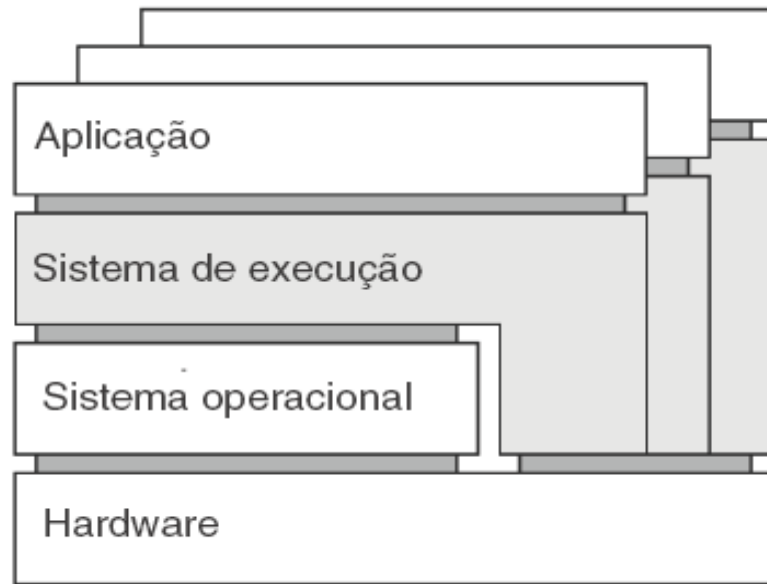
Execução de aplicações Windows no Linux Ubuntu

Monitor de máquina virtual

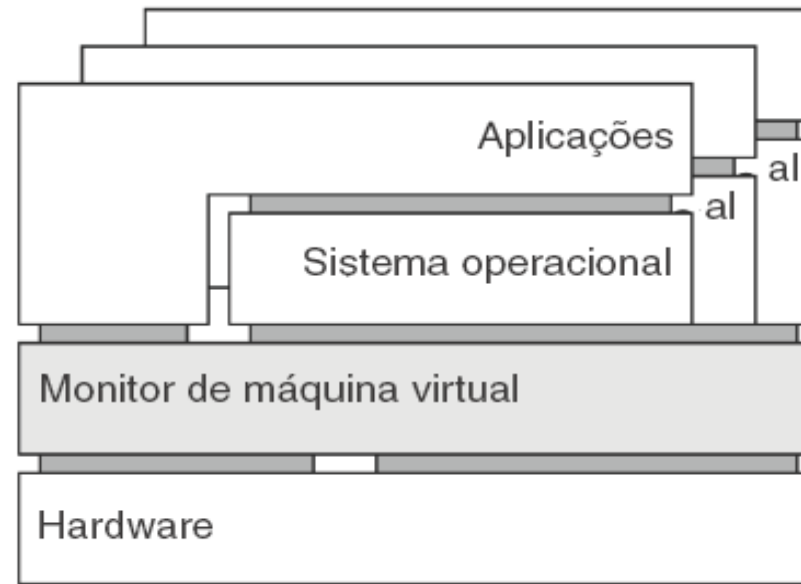
- Fornece o conjunto de instruções completo do hardware
- Vários sistemas operacionais diferentes executando independente e concorrentemente na mesma plataforma
- Segurança: isolamento de uma aplicação e seu ambiente
- Falhas não afetam a máquina inteira.

Exemplos: VMWare e VirtualBox

Virtualização



(a)



(b)

Figura 3.6 (a) Máquina virtual de processo, com várias instâncias de combinações (aplicação, execução).
(b) Monitor de máquina virtual com várias instâncias de combinações (aplicações, sistema operacional).

Concluindo

Foram abordados nesta aula:

- **Processos distribuídos**

Estes slides estão baseados na bibliografia

**Sistemas Distribuídos de TANENBAUM, A. S. e STEEN, M. V.
(2ª edição)**

Na próxima aula veremos:

- **Implementações dos processos distribuídos**

AULA Nº 05

SISTEMAS DISTRIBUÍDOS

Processos Distribuídos

Os slides são baseados no livro de Sistemas Distribuídos de TANENBAUM, A. S. e STEEN, M. V.